

Gaussian miniCaR - Gaussian Splatting for Object Detection using 4D Radar and Camera

Gunnar Martens
g.o.martens@student.tudelft.nl

Dylan Townsend
d.s.townsend@student.tudelft.nl

Koen Vogels
k.p.j.vogels@student.tudelft.nl

Abstract

LiDAR-based perception is widely used in autonomous vehicles, but its high cost and sensitivity to adverse weather motivate radar-camera fusion as a practical alternative. We present Gaussian miniCaR, a model that fuses 4D radar with monocular camera data by representing both modalities as Gaussian-splatted Bird's Eye View (BEV) maps. The radar encoder uses a Point Gaussian Encoder with local and global feature aggregation, while the camera encoder leverages depth estimation with auxiliary supervision from LiDAR during training. Both modalities are fused using a Cross-Modal Fusion (CMX) block and passed to a Center-Point detection backbone. On the View of Delft dataset, our approach achieves a 7.6 mAP improvement in the full evaluation area and a 2.1 mAP improvement in the driving corridor compared to the radar-only CenterPoint baseline. The model trains in under 4 hours on a shared A100 GPU. Our results demonstrate that Gaussian splatting enables effective multi-modal fusion and improves radar-based perception with room for improvement. The code is available at github.com/martensgun/AMP_FA.

1. Introduction

With the recent advancements in deep learning object detection architectures for autonomous vehicles (AVs), the potential for radar based object detection increases. This eliminates the need for expensive LiDAR sensors that suffer from adverse weather conditions [4], opening the pathway to economically viable AV production. A lightweight radar based CenterPoint baseline model [20] uses pillarization to project radar data into Bird's Eye View, allowing for multiscale spatial processing using a SECOND backbone and Feature Pyramid Neck (FPN). However, this baseline suffers from empty pillars and lacks semantic scene information. We propose **Gaussian miniCaR**, a model inspired by [11] that improves upon the CenterPoint baseline by fus-

ing monocular camera data with 4D radar data using Gaussian splatted BEV maps. Our main contributions are as follows:

- We replace the radar voxel encoder from the baseline model with a radar to Gaussian BEV map encoder, eliminating the issue of empty pillars.
- Monocular camera data is encoded into a Gaussian splatted BEV map, using LiDAR data during training for depth estimation. This map is fused with the radar map and passed to the baseline backbone, adding semantic information to the driving corridor region of interest (ROI).
- Our model achieves a 2.1% mAP increase in the driving corridor area over the baseline CenterPoint model on the View of Delft (VoD) [12] benchmark server, training in under four hours on the DelftBlue supercomputer using one A100-GPU and two CPUs with a total of 8 GB of memory.

2. Related Work

In the following section, we discuss the different aspects of related work including current practices of fusing camera and radar, the use of Gaussian Splatting in autonomous driving, and the methods of data enrichment.

2.1. Camera & Radar Fusion

Since LiDAR data is prone to poor performance in adverse weather conditions, the combination of radar and camera data is a promising approach to achieve robust perception in autonomous driving. As mentioned in [15], the fusion of the two modalities can be done early, on a feature level, or late, on a decision level. While each of them has its own advantages and disadvantages, early and feature-level fusion approaches have gained more popularity recently, largely attributed to the recent use of attention mechanisms [16]. This is visible in [1], where cross attention is used to fuse the two modalities. In [8], a Siamese network is used to fuse the features together and in [17], the fusion

is done in a sparse representation, which is more efficient for real-time applications.

2.2. Gaussian Splatting in Autonomous Driving

Since the development of Gaussian Splatting [6], many engineering fields have adopted this method to address domain specific challenges. In autonomous driving, Gaussian Splatting is incorporated to estimate depth from monocular cameras [10] or as a feature space to fuse different modalities together [11]. The different sensors are processed separately and then fused together at a later stage in BEV Gaussian feature space. In addition to the projection of features, Gaussian Splatting can also be used to characterize the inherent uncertainty of radar data, which is notorious for being noisy, as shown in [18] with different processing scopes.

2.3. Data Enrichment

Although 3+1D radars are becoming increasingly popular in autonomous driving, their point cloud density is still very sparse relative to LiDAR data, which increases the need for data enrichment or other techniques to obtain a higher performance of radar-based detectors. [2] and [13] use LiDAR data during training with different techniques (Knowledge and Feature Distillation) to improve the performance of a model without LiDAR. [19] proposes a teacher-student scheme, where the teacher model is trained with LiDAR data, and the student model learns to mimic the teacher’s output using only radar data.

3. Methodology

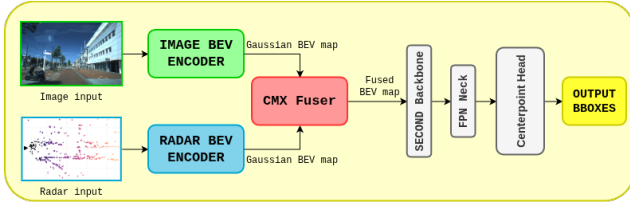


Figure 1. Schematic of the full detection pipeline. Our main contributions compared to the baseline are the radar and image encoders, as well as the CMX fuser.

3.1. Overview

The two main objectives of **Gaussian miniCaR** to improve upon the baseline radar CenterPoint model are as follows:

1. Encoding the radar data as a Gaussian splatted BEV map.
2. Encoding monocular front-facing camera data using a Gaussian-Lift-Splat-Shoot [10] approach in a Gaussian splatted BEV map.

We chose to focus on the Gaussian representation of the data streams, as Gaussians can capture general environment noise and the emergence of the Gaussian Splatting method poses a novel opportunity to process measurements in a more natural way and make fusion in the BEV more efficient. In leveraging the near-field information provided by the camera modality, we enhance the driving corridor feature map. The Gaussian BEV feature maps of both modalities are combined into a single feature map by a CMX type fuser [21] and passed to the original baseline backbone, as shown in Fig. 1. Our end-to-end model takes as input one radar point cloud containing N points and F features, resulting in an input $R \in \mathbb{R}^{N \times F}$. The second input is one RGB image, with height H and width W pixels, giving input $P \in \mathbb{R}^{3 \times H \times W}$. The model outputs a dictionary containing bounding boxes for the classes Pedestrian, Cyclist and Car, where each bounding box has the format (x, y, z, l, w, h, ψ) . These denote 3D position, length, width, height and yaw. Sections 3.2, 3.3 and 3.4 describe the radar point cloud encoder, image encoder and fusion of the two respective modalities. Section 3.5 describes our added primary and auxiliary (AUX) losses.

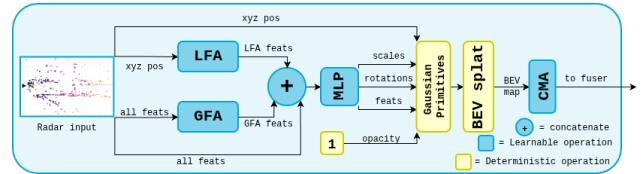


Figure 2. Schematic of the radar to Gaussian BEV encoder

3.2. Radar Branch

For encoding the radar point clouds into Gaussians with their respective features, we largely use the Point Gaussian Encoder from [18], as visible in Fig. 2. This combines a Local Feature Aggregator (LFA), which pulls features from local relationships using a small PointNet block [14] with a Global Feature Aggregator (GFA), which uses multi-head attention similar to [16] and normal Feedforward networks to aggregate features over the whole point cloud scale. These two sub-branches are concatenated together with the original radar features and transformed into the Gaussian primitive features of *scale*, *rotation* and *semantic features* using a simple Multilayer Perceptron (MLP). We take the original radar point positions for the Gaussians and set their opacity to 1. The Gaussians are splatted onto a BEV feature map using the code from the 3D Gaussian Splatting implementation [6]. This map goes through a Cross-Modality Alignment (CMA) block [2]. The CMA block densifies the BEV map features further by using a combination of up-sampling, downsampling and aggregation.

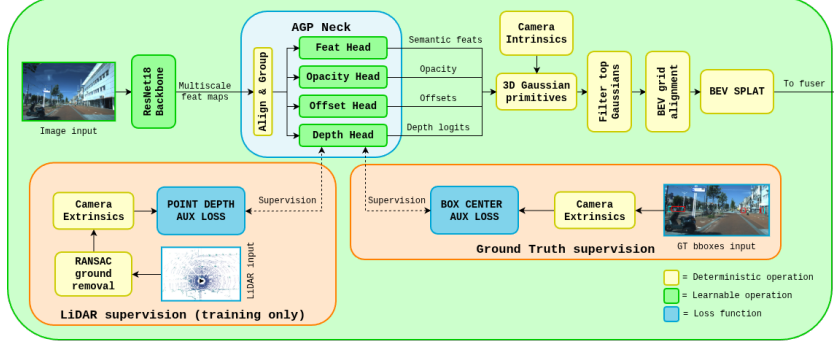


Figure 3. Schematic of the image to Gaussian BEV encoder

3.3. Image Branch

Before use, we preprocess the images by removing the top 25% portion followed by CNN downsampling (160 by 320 pixels) to reduce computation and bias the encoder towards road-related content. Our image encoder, illustrated in the top portion of Fig. 3, takes the image and passes it through a ResNet18 backbone pretrained on the ImageNet dataset [3] to obtain feature maps at multiple scales. These are *Aligned, Grouped* and *Projected* in the AGP neck, which contains four separate convolutional heads to predict semantic features (96 dimensions), opacities, offsets and depth logits z for a predefined number of depth bins, $M = 20$. The model samples bins uniformly in inverse-depth space, which results in a higher spatial resolution closer to the camera. Softmax is applied pixel-wise on the depth logits to produce an expected depth and variance (further explained in Section 3.5). These are used in combination with the camera’s intrinsic projection matrix to calculate 3D Gaussian primitives. This process is executed using raycasting techniques from [10]. The resulting Gaussians are filtered by the ROI ($x \in [0, 25]$, $y \in [-4, 4]$), covariance and border pixels. Based on the calculated confidence C (Sec. 3.5), a minimum threshold of 0.02 is used. We then performed stratified sampling of a 10 by 10 grid of the ROI to obtain the of K 3D Gaussians. We do this to remove low quality, noisy Gaussians from the data. Finally, we splat them onto the BEV plane after which they are passed to the fuser.

3.4. Fusing

Both aligned Gaussian BEV maps are fused on a single scale using the CMX-based fuser described in [11, 21]. The resulting fused features are afterwards passed to the rest of the network, where the architecture has not changed and is the same as in [7].

3.5. Losses

As images naturally lack depth information, we introduce two AUX losses for training the image encoder which

use data from the LiDAR point cloud and the ground truth (GT) bounding boxes. These are referred to as the Point Depth and Box Center loss as described in the lower portion of Fig. 3.

We first pre-process and cache the LiDAR data by removing ground points determined by a plane fitted with the RANSAC algorithm.

For the Point Depth loss, the depth logit z_i (for a single pixel) from the depth head is converted into a probability distribution $P(d_i)$ using softmax. Using the bin values B_i , a weighted sum is taken to predict the depth for that pixel $D_{pred} = \sum_{i=1}^M P(d_i)B_i$. The base loss for every pixel (u, v) is calculated with the smooth L1 loss function, eq. 1:

$$L_{base}^{(u,v)} = \text{SmoothL1}(D_{pred}^{(u,v)}, D_{gt}^{(u,v)}) \quad (1)$$

Here, D_{gt} is the GT depth from the processed point cloud. We also assign a peak confidence $C = \max_i P(d_i)$ to the pixel, coming from the highest probability bin. We calculate normalized entropy over all bins using eq. 2, to measure how ‘spread out’ the bins are.

$$H_{norm} = \frac{-\sum_{i=1}^M P(d_i) \log(P(d_i))}{\log(M)} \quad (2)$$

Confidence C is penalized by high entropy, calculating effective weight $W = C \cdot \max(0, 1 - \lambda H_{norm})$, where λ is tunable. The final loss is now calculated by the weighted average of the base loss L_{base} , normalized by the sum of the weights for all pixels V in eq. 3

$$L_{depth} = \frac{\sum_{(u,v) \in V} L_{base}^{(u,v)} W^{(u,v)}}{\sum_{(u,v) \in V} W^{(u,v)}} \quad (3)$$

The Box Center Loss L_{box} is calculated in the same way, except the GT depths D_{gt} come from the GT bounding box center of gravity.

To improve training of the radar encoder, we also introduce the Box Gaussian loss L_{bgl} . This loss is calculated by converting the predicted and GT bounding boxes into

Gaussian distributions $\hat{g} = \mathcal{N}(\hat{\mu}; \hat{\Sigma})$ and $g = \mathcal{N}(\mu; \Sigma)$ respectively. We then calculate the final loss using the KL divergence, eq. 4.

$$L_{bgl} = \frac{1}{N_b} \sum_{i=1}^{N_b} \text{KL}(\hat{g}_i, g_i) \quad (4)$$

Here, N_b is the number of GT bounding boxes. Refer to [18] for a detailed description. Together with the box loss L_b and heatmap loss L_h from the baseline model, and weights $\lambda_{1,2,3}$, the total head loss L_{head} is calculated with eq. 5:

$$L_{head} = \lambda_1 L_b + \lambda_2 L_h + \lambda_3 L_{bgl} \quad (5)$$

3.6. Multi-stage Training

We use a three stage approach to train the multi-modal Gaussian detection module. Firstly, the radar encoder and detection head are trained while the image encoder is disabled to establish a strong object detection baseline. Secondly, the image branch is trained while the radar branch and detection head are frozen, to learn depth and geometric cues without destabilizing the detector. Lastly, all models are unfrozen, and the complete system is trained end-to-end allowing the detection head to utilize the additional Gaussian modality.

4. Experiments

4.1. Dataset

We evaluate our model on the VoD dataset [12]. It contains 8693 frames of LiDAR, (stereo) camera, and 3+1D radar data in an urban setting. The model is trained on LiDAR, radar and monocular camera and evaluated without the LiDAR. The official dataset splits are used for the conducted experiments. The evaluation metrics are calculated for the entire annotated area (EAA) and the ROI corresponding to the driving corridor. Ablation studies are performed on the validation set.

4.2. Implementation Details

Our work is based on the Pointpillars model [7], where we replace the voxel encoder with multimodal encoders and a fusion module. The voxel size is set to $[0.16, 0.16, 5]$ and the resolution of the BEV map is 320×320 . The Gaussian features are set to 64 and the model is trained on a shared A100 GPU with two 4 GB memory CPUs, amounting to 8 GB total memory. Training is performed using the AdamW optimizer [9] with a weight decay of 0.05. We employ a learning rate scheduling strategy with a maximum learning rate of $8e-4$, a warm-up phase, and cosine decay.

We train the model for a total of 14 epochs with a combined runtime of 3 hours and 40 minutes:

1. Radar-only warmup (5 epochs): radar encoder and detection head train; image encoder disabled.
2. Image-only warmup (3 epochs): image encoder trains with auxiliary depth losses; radar encoder and detection head frozen.
3. End-to-end training: all modules unfrozen and optimized jointly.

The limited training time available poses a challenge as the end-to-end model must simultaneously optimize three coupled objectives: (i) stable radar-only detection, (ii) monocular depth estimation from noisy early predictions, and (iii) cross-modal fusion. We observe that auxiliary depth losses fluctuate significantly before stabilizing with minimal learning, and the warmup/freeze schedule delays end-to-end co-adaptation.

4.3. Ablation Study

Table 1 reports the model performance obtained by swapping and adding modules on the Centerpoint baseline (a) to the end-to-end model (d). The Point Gaussian Encoder (b) from [18] improves the mAP_{ROI} only slightly (0.4), while the mAP_{EAA} is increased significantly (7.6), which mirrors the findings in [18].

	Configuration	mAP_{EAA}	mAP_{ROI}
a)	baseline (Centerpoint)	27.9	53.0
b)	Radar	34.5	53.4
c)	Radar + CMA	32.6	55.0
d)	Radar + CMA + Image	32.8	55.1

Table 1. Ablation studies of components on VoD [12]. Bold text indicates best performance.

The image module had only a very minor effect on the overall performance of the detector, mainly due to the aforementioned issues during training. The general architecture however has the potential to achieve better results as laid out in [5, 11].

5. Conclusion

In this paper, we set out to improve upon the radar baseline detection model through multi-modal sensor fusion using a common Gaussian representation. Our results demonstrate that Gaussian splatting is a promising representation for multi-modal fusion in autonomous driving. However, the image encoder remains sensitive to depth-prediction stability, Gaussian filtering, and fusion alignment. Future work includes improving depth supervision, leveraging pretrained monocular depth models, incorporating uncertainty-aware fusion, and utilizing more compute resources to further enhance robustness.

6. Contributions

- Gunnar developed the radar encoder branch, optimized the training of that sub-module and wrote the sections 2 & 4.
- Dylan developed a standalone image encoder and merged branches to generate the end-to-end detector, and helped to write associated sections.
- Koen developed a data enrichment module, that was only partially used in the final model, created the figures and wrote the sections 1, 3 & 5.

References

- [1] Xiaokai Bai, Zhu Yu, Lianqing Zheng, Xiaohan Zhang, Zili Zhou, Xue Zhang, Fang Wang, Jie Bai, and Hui-Liang Shen. Sgdet3d: Semantics and geometry fusion for 3d object detection using 4d radar and camera. *IEEE Robotics and Automation Letters*, 10(1):828–835, 2025. 1
- [2] Geonho Bang, Kwangjin Choi, Jisong Kim, Dongsuk Kum, and Jun Won Choi. Radardistill: Boosting radar-based object detection performance via knowledge distillation from lidar features, 2025. 2
- [3] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009. 3
- [4] Mariella Dreissig, Dominik Scheuble, Florian Piewak, and Joschka Boedecker. Survey on lidar perception in adverse weather conditions, 2023. 1
- [5] Wanshui Gan, Fang Liu, Hongbin Xu, Ningkai Mo, and Naoto Yokoya. Gaussianocc: Fully self-supervised and efficient 3d occupancy estimation with gaussian splatting, 2025. 4
- [6] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering, 2023. 2
- [7] Alex H Lang, Sourabh Vora, Holger Caesar, Lubing Zhou, Jiong Yang, and Oscar Beijbom. Pointpillars: Fast encoders for object detection from point clouds. In *CVPR*, 2019. 3, 4
- [8] H. Liu, C. Wu, and H. Wang. Real time object detection using LiDAR and camera fusion for autonomous driving. *Scientific Reports*, 13:8056, 2023. 1
- [9] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019. 4
- [10] Shu-Wei Lu, Yi-Hsuan Tsai, and Yi-Ting Chen. Toward real-world bev perception: Depth uncertainty estimation via gaussian splatting, 2025. 2, 3
- [11] Santiago Montiel-Marín, Miguel Antunes-García, Fabio Sánchez-García, Angel Llamazares, Holger Caesar, and Luis M. Bergasa. Gaussiancar: Gaussian splatting for efficient camera-radar fusion, 2026. 1, 2, 3, 4
- [12] Andras Palffy, Ewoud Pool, Srimannarayana Baratam, Julian F. P. Kooij, and Darius M. Gavrilă. Multi-class road user detection with 3+1d radar in the view-of-delft dataset. *IEEE Robotics and Automation Letters*, 7(2):4961–4968, 2022. 1, 4
- [13] Patrick Palmer, Martin Krüger, Stefan Schütte, Richard Altendorfer, Ganesh Adam, and Torsten Bertram. Lerojd: Lidar extended radar-only object detection, 2024. 2
- [14] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation, 2017. 2
- [15] Arvind Srivastav and Soumyajit Mandal. Radars for autonomous driving: A review of deep learning methods and challenges. *IEEE Access*, 11:97147–97168, 2023. 1
- [16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. 1, 2
- [17] Philipp Wolters, Johannes Gilg, Torben Teepe, Fabian Herzog, Felix Fent, and Gerhard Rigoll. Sparc: Sparse radar-camera fusion for 3d object detection, 2025. 1
- [18] Weiyi Xiong, Bing Zhu, and Zewei Zheng. Radargaussian-det3d: Gaussian representation-based real-time 3d object detection with 4d automotive radars. *IEEE Robotics and Automation Letters*, 11(5):5709–5716, May 2026. 2, 4
- [19] Ruoyu Xu, Zhiyu Xiang, Chenwei Zhang, Hanzhi Zhong, Xijun Zhao, Ruina Dang, Peng Xu, Tianyu Pu, and Eryun Liu. Sckd: Semi-supervised cross-modality knowledge distillation for 4d radar object detection, 2024. 2
- [20] Tianwei Yin, Xingyi Zhou, and Philipp Krähenbühl. Center-based 3d object detection and tracking, 2021. 1
- [21] Jiaming Zhang, Huayao Liu, Kailun Yang, Xinxin Hu, Ruiping Liu, and Rainer Stiefelhagen. Cmx: Cross-modal fusion for rgb-x semantic segmentation with transformers, 2023. 2, 3